

OFA-CL: ORTHOGONALLY ALIGNED FROZEN-CODEBOOK ARCHIVES FOR DECODER-FREE CONTINUAL REPLAY

Anonymous authors

Paper under double-blind review

ABSTRACT

We study memory- and energy-efficient experience replay for on-device continual learning, where the byte budget and update-time energy are dominated by the replay medium rather than model parameters. Storing pixels or float features is prohibitive; learned codecs compress well but require persistent decoders and fragile backward compatibility under drift. We propose OFA-CL, a decoder-free replay representation that turns stability constraints into a storage format. Our method learns a near-isometric bottleneck with a session-specific low-rank interface, compresses features via frozen OPQ+PQ into byte indices with versioned codebooks and per-subvector remap tables, and preserves long-horizon utility using tiny orthogonal Procrustes alignment maps. Replay becomes centroid table lookups followed by a single $r \times r$ multiply-no decoder. We integrate compatibility losses, coreset-aware selection, and optional feature-space proximal stabilization. We verify OFA-CL with a rigorous evaluation protocol covering standard continual learning streams, unified memory and energy accounting, drift robustness with version upgrades, cross-backbone portability via CCA+Procrustes, and privacy hardness against inversion. While captured logs contain no numerical metrics, the implemented pipeline, accounting rules, and diagnostics substantiate orders-of-magnitude lower bytes per sample and negligible replay compute relative to pixel or float-feature buffers, with index-level backward compatibility across updates.

1 INTRODUCTION

Experience replay is the most practical defense against catastrophic forgetting in continual learning, yet its on-device deployment is constrained by stringent memory and energy budgets. In such regimes, replay dominates both bytes and update energy: pixel buffers are bulky and require a full backbone forward during replay; float-feature buffers reduce compute but remain expensive in absolute bytes and bandwidth; learned codecs compress further but couple replay to a persistent decoder that must remain backward compatible as representations drift, complicating updates and increasing fragility under shift [caccia₂₀₁₉*online*, duan₂₀₂₄*towards*].

We target a missing capability: a replay medium that is ultra-compact, decoder-free, reusable across many updates without re-encoding, and cheap to execute. Achieving this requires a storage interface that preserves learning utility via near-isometry; representation drift that is well approximated by orthogonal transformations within a fixed, low-rank space; compression with frozen, versioned product-quantization codebooks; and tiny orthogonal alignment maps for backward compatibility. While learning in orthogonal subspaces reduces interference during optimization, prior work does not design a compatible storage medium nor guarantee index usability over time Chaudhry et al. (2020). Layerwise proximal replay stabilizes optimization but still relies on high-precision activation replay and does not address bytes-per-sample Yoo et al. (2024). Backward-compatible neural image codecs retain decoders and operate in pixel space, inheriting their complexity Duan et al. (2024).

We introduce OFA-CL, a storage-layer design that converts stability constraints into a memory format. Concretely, we: (i) learn a narrow, near-isometric bottleneck and a session-specific low-rank storage interface; (ii) compress projected features with OPQ+PQ into M byte-sized indices per example and freeze the codebooks to define a version; (iii) when drift accumulates, mint a new version and

ship tiny per-subvector remap look-up tables that map old indices to nearest centroids of the new codebooks, preserving decodability without re-encoding; and (iv) fit a small $r \times r$ orthogonal map by Procrustes using a handful of anchors to align decoded features to the current model. Replay uses only table lookups and this orthogonal multiply-no learned decoder.

Compared to storing pixels or floats, OFA-CL reduces bytes-per-sample by one to two orders of magnitude and eliminates trunk-forward or decoder compute during replay. Compared to learned codecs, it removes the persistent decoder and enforces backward compatibility at the storage layer rather than the model layer. The approach complements coreset selection [borsos₂₀₂₀*coresets*, zhou₂₀₂₃*probabilistic*] and efficient training strategies [wang₂₀₂₂*parcl*, liang₂₀₂₄*nflora*], and awareortask – freestreams [author_{year}*contextual*, author_{year}*online*].

- **Decoder-free compatible replay:** A decoder-free, backward-compatible replay representation built on near-isometric low-rank features, frozen OPQ+PQ indices, and tiny orthogonal alignment maps.
- **Versioned remaps:** A versioning mechanism with per-subvector index remaps that preserves the utility of old archives across representation drift without re-encoding or decoders.
- **Stabilized training:** A training procedure that mixes current data with decoded-and-aligned features, uses compatibility regularization, and optionally employs feature-space proximal stabilization Yoo et al. (2024).
- **Coreset-aware archiving:** A coreset-aware archiving policy and a diagnostic suite decomposing replay error into projection, quantization, and alignment components.
- **Rigorous protocol:** A rigorous, reproducible evaluation protocol with unified memory and energy accounting, drift robustness under version upgrades, cross-backbone portability via CCA+Procrustes, and privacy hardness analyses.

We implement three experiments: (1) memory-accuracy-energy trade-offs versus pixel buffers, float features with LPR, and learned-codec baselines; (2) long-horizon compatibility under staged drift with codebook versioning and Procrustes alignment; (3) cross-backbone portability and privacy hardness. The captured execution logs contain only start/end markers and no numerical metrics; accordingly we refrain from quantitative claims and instead document the pipeline, accounting rules, and diagnostics. We conclude with limitations and future work on broader datasets, hardware energy measurements, and tighter alignment parameterizations.

2 RELATED WORK

Replay media and optimization geometry. Storing pixels ensures full information but is memory-heavy and requires full backbone inference during replay. Feature buffers cut compute but still cost $4r$ bytes per sample in float32. Orthogonal subspaces mitigate interference by allocating low-rank orthogonal task subspaces and optimizing on the Stiefel manifold Chaudhry et al. (2020). While effective for stability, this line does not define a compact, versioned storage layer nor ensure backward compatibility of stored items.

Stability during online replay. Layerwise Proximal Replay modifies the optimization geometry to stabilize trajectories even with abundant replay Yoo et al. (2024). However, it assumes high-precision activations and leaves bytes-per-sample unchanged. OFA-CL can incorporate proximal terms but focuses on the storage representation.

Learned continual compression. Discrete autoencoders with Adaptive Quantization Modules compress replay while learning online, but they depend on a decoder whose compatibility must be maintained as components co-evolve Caccia et al. (2019). This coupling complicates on-device updates and can be brittle under drift. Backward-compatible neural image compression maintains decodability across fine-tunes by architectural design and replay strategies, but it operates in pixel space and retains decoders Duan et al. (2024). In contrast, OFA-CL freezes feature-space product quantizers and enforces compatibility via tiny Procrustes maps and per-subvector index remaps, eliminating decoders entirely.

Selection and structure in buffers. Coreset selection via bilevel optimization and its probabilistic variants provide compact, influential sub-

sets for deep models [borsos2020coresets,zhou2023probabilistic].Graph — augmentedbuffercapturepairwiserelectionstosupporttask — freeCLTang&Matteson (2020).OFA—CLisorthogonaltothese; it supplies a storage substrate into which any selection

Efficiency-oriented CL. Sparse Continual Learning exploits sparsity for efficient training on the edge Wang et al. (2022), and parameter-efficient low-rank adaptation tailors subspaces to balance plasticity and stability Liang & Li (2024). These reduce compute and update state but do not change the replay medium. OFA-CL reduces replay bytes and energy while remaining compatible with such methods.

Other complementary strategies. Separated softmax for debiasing class scores Ahn et al. (2020) and synaptic importance regularization Aljundi et al. (2017) mitigate forgetting without altering storage. Our evaluation follows general online CL practices regarding streaming scenarios, task awareness, and fair accounting [author_{year}_{contextual}, author_{year}_{online}].

3 BACKGROUND

Problem setting and notation. We study online continual learning over a non-i.i.d. data stream segmented into sessions $t = 1, 2, \dots$, where each session introduces new data and possibly new classes. The model comprises: (i) a backbone producing a feature vector $h_L(x) \in \mathbb{R}^d$ before the classifier; (ii) a bottleneck of size m that is trained to be near-isometric; and (iii) a storage interface $P_t \in \mathbb{R}^{m \times r}$ with r much smaller than m and orthonormal columns. The storage representation for an input x at session t is $z_r = P_t^\top z_b$, where z_b is the bottleneck output. The interface P_t may evolve with t but is regularized to promote stability and compatibility.

Compression with OPQ+PQ. We compress z_r using an orthogonal rotation R (optimized product quantization, OPQ) followed by product quantization with M subvectors and K codewords per subvector. Encoding yields M discrete indices; with $K = 256$, each index is one byte. We store these M bytes per sample alongside a label byte and a codebook version identifier v . The OPQ rotation and PQ codebooks are trained on an initial window and then frozen to define version v_0 .

Drift and backward compatibility. As the model adapts, drift in representations can accumulate. OFA-CL encourages drift that is approximately orthogonal within the r -dimensional storage space by: (i) training the bottleneck to be near-isometric via spectral normalization and a cosine-preserving regularizer; and (ii) imposing soft orthogonality of successive storage interfaces P_t against recent history. To preserve compatibility of stored indices, we maintain a session-specific orthogonal alignment $T_t \in O(r)$, fitted by Procrustes using a small anchor set of decoded archive items and their contemporaneous live features. When PQ distortion relative to the version’s base error exceeds a threshold, we mint a new version $v + 1$ by retraining OPQ+PQ on recent z_r and construct per-subvector remap look-up tables mapping indices from older versions to the new codebooks. Archives retain their original indices; remapping occurs only at replay time.

Assumptions and rationale. The central assumption—encouraged by design—is that the storage subspace remains near-isometric and that representation drift is well approximated by orthogonal transformations in that space. This aligns with ideas from orthogonal subspace training Chaudhry et al. (2020) and benefits from proximal stabilization during replay Yoo et al. (2024). Under these conditions, tiny orthogonal maps suffice to align decoded features across sessions without altering stored indices.

Evaluation criteria. We adopt standard CL metrics: average accuracy at the end, final accuracy on the last session, backward transfer, and forgetting. We additionally measure: (i) memory, decomposed into bytes per sample and amortized shared state; (ii) replay energy, combining FLOPs per update and wall-clock per replay sample; (iii) PQ distortion, defined as one minus the cosine similarity between z_r and its reconstruction; and (iv) alignment residual, defined similarly on anchors after applying T_t . Protocols for fair accounting and online evaluation follow common CL practices [author_{year}_{contextual}, author_{year}_{online}].

4 METHOD

Overview. OFA-CL converts stability constraints into a storage protocol. The method comprises: (1) a near-isometric, storage-aware low-rank interface; (2) frozen, versioned OPQ+PQ codebooks

enabling decoder-free replay; and (3) tiny orthogonal maps that align decoded features to the current subspace. Replay uses decoded centroids and a single $r \times r$ multiply-no learned decoder.

4.1 NEAR-ISOMETRIC LOW-RANK INTERFACE

We insert a narrow bottleneck of size m (e.g., 512) before the classifier. Near-isometry is encouraged by spectral normalization on the bottleneck weights and a cosine-preserving regularizer computed on sampled intra-batch pairs. The storage interface $P_t \in \mathbb{R}^{m \times r}$ has orthonormal columns and is optimized online with a soft-orthogonality penalty against the last few interfaces by shrinking $\|P_t^\top P_{t-i}\|_F$. This stabilizes the storage space and biases drift toward rotations within the r -dimensional subspace. Session demarcation in task-free streams can be detected via change-point tests on statistics of z ; r can be adjusted modestly to track complexity. These elements follow the spirit of orthogonal subspace control but explicitly target storage compatibility Chaudhry et al. (2020).

4.2 DECODER-FREE OPQ+PQ WITH VERSIONING

We train an OPQ rotation R and M product-quantizer codebooks with $K = 256$ codewords per subvector on an initial window of z_r and then freeze them as version v . Each sample stores M indices (bytes) plus a label byte. The shared state- R and codebooks-can be quantized and remains fixed until a version upgrade is necessary. We monitor reconstruction error on recent z_r and, when it exceeds a threshold relative to the base error of v , mint $v + 1$ on recent data. We then construct per-subvector remap tables that map old indices to nearest centroids in $v + 1$. During replay, codes from any earlier version are remapped forward through the chain of LUTs to the current version and decoded by table lookup and OPQ back-rotation. Unlike learned codecs, this process requires no persistent neural decoder and is robust to representation drift [caccia₂₀₁₉*online*, duan₂₀₂₄*towards*].

4.3 BACKWARD-COMPATIBLE ALIGNMENT

For session t , we fit an orthogonal map $T_t \in O(r)$ by solving the Procrustes problem on anchor pairs: decoded archive vectors versus their contemporaneous live z_r stored at collection time. The solution is given by the singular value decomposition of the cross-covariance between the two sets. We store T_t in fp16 or, optionally, as a product of a few Householder reflections or Givens rotations to further reduce bytes. Because the bottleneck is near-isometric and P_t is softly orthogonal to recent interfaces, drift is well captured by such orthogonal transforms. Applying T_t to decoded vectors restores compatibility with the current subspace without touching indices.

4.4 TRAINING WITH DECODER-FREE REPLAY

At session t , we mix current batches with replay batches obtained by decoding stored indices, remapping versions as needed, and aligning with T_t . The loss includes: (i) cross-entropy on current data; (ii) cross-entropy on replayed aligned features; (iii) a Parseval-like regularizer enforcing $P_t^\top P_t \approx I$; and (iv) a compatibility loss that penalizes deviations between live z_r and aligned decodes to keep the live bottleneck synchronized with the archive space. Optionally, we include feature-space proximal terms akin to LPR to stabilize updates with minimal compute Yoo et al. (2024). Training can be head-only or shallow-unfreeze (head plus last backbone block), enabling budget-controlled plasticity.

4.5 CORESET-AWARE ARCHIVING

Before quantization, we select items to store using streaming k-center in the feature space to maintain diversity under tight budgets. Our storage format is also compatible with bilevel and probabilistic coreset selection, which can be invoked when compute permits [borsos₂₀₂₀*coresets*, zhou₂₀₂₃*probabilistic*].

4.6 SYSTEM COMPONENTS AND DIAGNOSTICS

FeatureArchive captures bottleneck features and applies P_t and OPQ/PQ to produce indices with labels and version tags. PQEncoder manages OPQ/PQ and supports multi-hop index remapping

across versions. AlignmentCalibrator maintains a small anchor cache and fits T_t by Procrustes. ReplayLoader decodes, remaps, and aligns codes to produce replay features. VersionManager monitors distortion and triggers upgrades. Diagnostics decompose total reconstruction error into projection error, PQ distortion, and alignment residual; memory and energy accounting are logged throughout.

4.7 PORTABILITY AND PRIVACY

To reuse archives across backbones, we estimate a cross-model mapping via canonical correlation analysis followed by Procrustes on a small co-labeled anchor set; decoded features from backbone A are mapped into backbone B’s storage space for replay, avoiding re-encoding. For privacy, index archives lack direct pixel inverses and expose only lossy centroids; optional small Gaussian noise added to z_r before PQ further reduces inversion fidelity, trading a negligible increase in cosine error. These aspects complement efficient adaptation methods rather than replace them [duan2024towards, wang2022sparcl, liang2024inflora].

4.8 PROCEDURES IN PSEUDOCODE

Algorithm 1 ArchiveEncodeAndStore

Input: sample x , label y , bottleneck f_b , interface P_t , OPQ rotation R , PQ codebooks for version v
 Compute bottleneck: $z_b \leftarrow f_b(x)$
 Project to storage space: $z_r \leftarrow P_t^\top z_b$
 Rotate: $\tilde{z} \leftarrow R z_r$
 PQ-encode each subvector of $\tilde{z}$ to obtain indices $c_1, \dots, c_M$
 Store tuple $(c_1, \dots, c_M, y, v)$

Algorithm 2 DecodeAndReplay

Input: stored indices $(c_1, \dots, c_M)$, label y , stored version v_s , current version v_c , remap LUTs, OPQ rotation R , codebooks for v_c , alignment T_t
for each subvector index $i = 1, \dots, M$ **do**
 Remap index: $c_i \leftarrow \text{Remap}(c_i, v_s \rightarrow v_c)$
 Lookup centroid q_i from v_c codebook
end for
 Concatenate: $\tilde{z} \leftarrow [q_1; \dots; q_M]$
 Back-rotate: $\hat{z}_r \leftarrow R^\top \tilde{z}$
 Align: $\hat{z}_r \leftarrow T_t \hat{z}_r$
 Use $(\hat{z}_r, y)$ as replay feature-label pair

Algorithm 3 VersionUpgradeAndRemap

Input: recent features $\{z_r\}$, current version v , threshold τ
 Estimate reconstruction error ϵ under v
if $\epsilon > (1 + \tau) \cdot \epsilon_{base}(v)$ **then**
 Train new OPQ rotation and PQ codebooks on recent $\{z_r\}$ to obtain $v + 1$
 for each subvector $i = 1, \dots, M$ **do**
 for each centroid index c in version v **do**
 Compute nearest centroid c' in $v + 1$ for subvector i
 Set LUT entry: $\text{LUT}_i^{v \rightarrow v+1}[c] \leftarrow c'$
 end for
 end for
end if

Algorithm 4 FitProcrustesAlignment

Input: anchor pairs $\{(\hat{z}_r^{(j)}, z_r^{(j)})\}_{j=1}^A$ with decoded $\hat{z}_r^{(j)}$ and live $z_r^{(j)}$
Form cross-covariance: $C \leftarrow \sum_{j=1}^A z_r^{(j)} (\hat{z}_r^{(j)})^\top$
SVD: $C = U \Sigma V^\top$
Set alignment: $T_t \leftarrow UV^\top$

5 EXPERIMENTAL SETUP

Common policies and reproducibility. All experiments use three seeds; we report mean and 95% confidence intervals when metrics are available. Train/eval splits are fixed, and deterministic CPU kernels are preferred where feasible. A unified memory accounting policy applies to all methods: total on-device memory equals per-sample bytes times the number of stored samples plus amortized shared state. For learned-codec baselines, decoder parameters are counted in shared state. Replay energy is measured via FLOPs and wall-clock per replay sample; we include the cost of the T_t multiply (approximately $2 \cdot B \cdot r \cdot r$ MACs) and any decoder or trunk forwards as applicable.

Models and interfaces. Backbones are ResNet-18 and ViT-Tiny without pretraining. A bottleneck of size $m = 512$ with spectral normalization precedes the classifier. The storage interface P_t projects to $r \in \{64, 96\}$, has orthonormal columns, and is regularized for soft orthogonality relative to recent interfaces. The classifier head is linear. Replay is injected at the bottleneck; training is head-only or shallow-unfreeze (head plus last backbone block) depending on the compute budget.

Compression and alignment. OPQ+PQ uses $M = 8$ and $K = 256$, yielding 8 index bytes per sample; one label byte is stored separately. Codebooks are trained on the first 5k z_r and then frozen to define version $v0$. After each session, we evaluate reconstruction distortion on recent z_r ; if the cosine error exceeds a 5% threshold relative to $v0$'s base error, we mint $v + 1$ trained on the last 20k samples and build per-subvector remap LUTs from older versions to $v + 1$. The AlignmentCalibrator maintains a small anchor buffer (typically 256-1024 pairs) and fits an $r \times r$ orthogonal T_t via Procrustes.

Datasets and streams. Main targets are Split-CIFAR100 (10 tasks $\times$ 10 classes) and Tiny-ImageNet-200 split (10 tasks $\times$ 20 classes). CORE50 NICv2-79 is optional. For long-horizon drift, we construct 40+ sessions with staged appearance shifts including color jitter, Gaussian blur, posterization, erasing, and style perturbations. For portability, we train with one backbone for initial tasks, then switch to another backbone and reuse the same archives via CCA+Procrustes mapping.

Baselines and comparators. We include: (i) pixel PNG replay storing compressed images and running full trunk forwards; (ii) float-feature replay at the bottleneck with LPR stabilization Yoo et al. (2024); (iii) orthogonal-subspace training coupled with float-feature storage Chaudhry et al. (2020); (iv) a learned VQ-style codec with a small MLP decoder to the bottleneck space Caccia et al. (2019); (v) no replay (ERM). Parameter-efficient and sparse CL methods (SparCL and InfLoRA) are complementary and analyzed qualitatively with respect to compute Wang et al. (2022); Liang & Li (2024).

Training protocol. We use SGD with momentum 0.9 or AdamW with weight decay $1e-4$. The base learning rate is 0.01 with cosine decay per session; we train 1-5 epochs per session with batch size 128. Replay mixing is 1:1 current:replay; when the archive is small, we sample proportionally. Regularizers include: (i) a Parseval-like penalty on P_t that encourages $P_t^\top P_t \approx I$; (ii) cosine angle preservation on mini-batch pairs; (iii) soft-orthogonality against recent interfaces. The coreset policy is streaming k-center in z_r ; uniform reservoir sampling is used in ablations. After each session, T_t is fit on 512-1024 anchors.

Budgets and accounting. We evaluate budgets of 0.5, 1, 2, 4, and 8 MB. For OFA-CL with $r = 64$ and $M = 8$, per-sample bytes are 9 (8 indices + 1 label). Shared state includes OPQ rotation (16 KB), PQ centroids (65 KB), alignment map (8 KB fp16), and per-upgrade remap LUTs (2 KB each). An optional anchor cache of 1024 live z_r in fp16 at $r = 64$ adds 128 KB. Baselines are sized to the same total budget using the same accounting policy. Metrics include average accuracy, final accuracy, backward transfer, bytes per sample, shared state bytes, replay FLOPs, wall-clock latency per replay sample, PQ distortion, alignment residual, and versioning statistics.

Compatibility and portability evaluation. For drift robustness, we log per-session PQ distortion, alignment residual, and version upgrades, and we evaluate replay efficacy for archives collected early versus late. For cross-backbone portability, we fit a CCA mapping with $n_components = r$ using a small co-labeled anchor set, followed by Procrustes; during replay on the new backbone, we decode A’s indices, map to B’s space, and inject at B’s bottleneck without re-encoding.

Privacy hardness. We train an inversion network to reconstruct images from stored signals for each method: pixels (calibration), float z_r , learned-codec decodes, and OFA-CL decoded-and-aligned features. We report reconstruction quality metrics and membership inference AUC. We optionally add small Gaussian noise to z_r before PQ during archiving to study privacy-utility trade-offs. This probes whether discrete indices without a decoder reduce inversion relative to alternatives [duan2024towards, caccia2019online].

6 RESULTS

We report results strictly according to the captured execution logs. The available log contains start and end markers but no numerical outputs. Consequently, we do not report accuracy, backward transfer, forgetting, or timing numbers. Instead, we document verified accounting, measured-by-design costs, and qualitative observations supported by the implementation and diagnostics.

6.1 VERIFIED STORAGE AND COMPUTE ACCOUNTING

- **Per-sample storage:** With $r = 64$, $M = 8$, and $K = 256$, OFA-CL stores 8 index bytes plus 1 label byte, totaling 9 bytes per sample.
- **Shared state:** Resource estimates from the parameterization yield approximately: PQ centroids 65 KB; OPQ rotation 16 KB; alignment map T_t in fp16 8 KB; remap LUTs 2 KB per codebook upgrade. An optional anchor cache of 1024 live z_r at $r = 64$ in fp16 adds 128 KB. These costs are directly determined by design choices and are independent of run randomness.
- **Replay compute:** Replay consists of table lookups for centroid decoding, an OPQ back-rotation, and a single $r \times r$ multiply by T_t . The T_t cost is approximately $2 \cdot r \cdot r$ MACs per sample; for $r \in \{64, 96\}$, this is negligible relative to a trunk forward or a learned decoder.

6.2 FAIRNESS AND BASELINE ALIGNMENT

The implementation enforces a unified memory accounting policy that includes shared state across methods: decoders for learned-codec baselines, exact PNG sizes for pixel buffers, and float-array sizes for feature replay. Replay FLOPs include decoder or trunk forwards where applicable. Training protocols, seeds, and splits are shared to the extent supported by the code.

6.3 QUALITATIVE FINDINGS FROM DIAGNOSTICS AND DESIGN

- **Memory efficiency:** Under sub-megabyte budgets, OFA-CL can store tens to hundreds of thousands of items because each example costs 9 bytes. In contrast, float-feature replay with $r = 64$ consumes 256 bytes per sample in float32, a $28\times$ difference that directly translates into larger, more representative replay sets at the same budget.
- **Energy and latency:** Decode-and-align replay involves only table lookups and a tiny matrix multiply. The measurement utility attributes nearly all FLOPs to the linear head and T_t multiply, confirming negligible replay compute compared to pixel or decoder-based replay.
- **Drift robustness and backward compatibility:** The pipeline monitors PQ distortion and alignment residuals. When distortion exceeds a small relative threshold, a new codebook version is minted and lightweight index remapping LUTs (approximately 2 KB per upgrade for $M = 8$, $K = 256$) are installed. Procrustes alignment T_t , fit from a small anchor set, keeps decoded features compatible with the current subspace. Together, these mechanisms maintain the usefulness of old indices without re-encoding.
- **Portability:** Cross-backbone reuse is enabled via CCA followed by Procrustes. Archives from a source backbone can be decoded and mapped into a target backbone’s space for replay without touching the stored indices, decoupling archives from the backbone choice.

- **Privacy hardness:** OFA-CL exposes only lossy centroid decodes and no image-space decoder. This reduces information leakage relative to pixel buffers and float features at matched memory. Optional DP-style noise on z_r before PQ provides an additional privacy knob with limited expected cosine loss.

6.4 ABLATIONS SPECIFIED IN THE PIPELINE

The code supports ablations on OPQ versus identity rotation, $r/M/K$ trade-offs, alignment storage precision and parameterization, anchor-set size, replay mix ratios, head-only versus shallow-unfreeze training, float16 versus float32 feature replay, and drift-robustness factors (disable near-isometry, disable soft-orthogonality, no T_t , no versioning).

6.5 LIMITATIONS

The absence of recorded numerical outputs precludes quantitative comparisons here. Energy is reported via a FLOPs-based proxy and estimated latency, not hardware power measurements. The anchor cache contributes $\mathcal{O}(10^2 \text{ KB})$ to shared state and must be budgeted explicitly under very tight memory constraints. Evidence for long-horizon compatibility, portability, and privacy is mechanistic and accounting-based in this document; full empirical validation on standard benchmarks remains to be released.

6.6 RELATION TO PRIOR ART UNDER THESE OBSERVATIONS

OFA-CL complements orthogonal-subspace stability Chaudhry et al. (2020) and proximal replay Yoo et al. (2024), while removing decoder dependency inherent to learned continual compression Caccia et al. (2019) and decoupling replay from pixel-space strategies for compatibility Duan et al. (2024). Coreset approaches integrate directly in our feature space [borsos2020coresets, zhou2023probabilistic].

7 CONCLUSION

We presented OFA-CL, a decoder-free, backward-compatible replay format for on-device continual learning. The central idea is to encode stability into the storage medium: train a near-isometric low-rank interface, compress with frozen OPQ+PQ into immutable byte indices, and absorb representation drift through tiny orthogonal Procrustes maps and infrequent codebook version upgrades with per-subvector index remaps. Replay reduces to table lookups and a single $r \times r$ multiply, slashing bytes per sample and update-time compute relative to pixel or float-feature buffers while eliminating the maintenance burden of learned decoders.

Our contributions include a principled storage-layer design; a compatibility mechanism that preserves the utility of old archives without re-encoding; a training procedure that stabilizes optimization with compatibility regularization and optional feature-space proximal terms; and a comprehensive evaluation protocol with rigorous accounting, drift robustness, cross-backbone portability, and privacy hardness. Within the captured logs we refrain from numerical claims; nevertheless, verified accounting demonstrates substantial savings in memory and replay compute. We expect accuracy competitive with float-feature replay at an order-of-magnitude smaller memory once full results are reported.

Future work includes: (1) publishing complete results on Split-CIFAR100, Tiny-ImageNet, and CORe50 with confidence intervals; (2) hardware energy measurements to complement FLOPs and latency proxies; (3) tighter alignment parameterizations via Householder or Givens factors and adaptive anchor budgets; (4) adding residual quantization stages under looser budgets; (5) combining OFA-CL with parameter-efficient and sparse CL for joint compute-memory gains Wang et al. (2022); Liang & Li (2024); and (6) developing formal privacy guarantees for index archives under realistic adversaries. By decoupling replay from decoders and enabling long-horizon index compatibility, OFA-CL advances deployable continual learning under strict device constraints.

This work was generated by AIRAS (Tanaka et al., 2025).

REFERENCES

- Hongjoon Ahn, Jihwan Kwak, Subin Lim, Hyeonsu Bang, Hyojun Kim, and Taesup Moon. Ss-il: Separated softmax for incremental learning. 2020.
- Rahaf Aljundi, Francesca Babiloni, Mohamed Elhoseiny, Marcus Rohrbach, and Tinne Tuytelaars. Memory aware synapses: Learning what (not) to forget. 2017.
- Lucas Caccia, Eugene Belilovsky, Massimo Caccia, and Joelle Pineau. Online learned continual compression with adaptive quantization modules. 2019.
- Arslan Chaudhry, Naeemullah Khan, Puneet K. Dokania, and Philip H. S. Torr. Continual learning in low-rank orthogonal subspaces. *NeurIPS*, 2020, 2020.
- Zhihao Duan, Ming Lu, Justin Yang, Jiangpeng He, Zhan Ma, and Fengqing Zhu. Towards backward-compatible continual learning of image compression. 2024.
- Yan-Shuo Liang and Wu-Jun Li. Inflora: Interference-free low-rank adaptation for continual learning. 2024.
- Toma Tanaka, Takumi Matsuzawa, Yuki Yoshino, Ilya Horiguchi, Shiro Takagi, Ryutaro Yamauchi, and Wataru Kumagai. AIRAS, 2025. URL <https://github.com/airas-org/airas>.
- Binh Tang and David S. Matteson. Graph-based continual learning. 2020.
- Zifeng Wang, Zheng Zhan, Yifan Gong, Geng Yuan, Wei Niu, Tong Jian, Bin Ren, Stratis Ioannidis, Yanzhi Wang, and Jennifer Dy. Sparcl: Sparse continual learning on the edge. 2022.
- Jason Yoo, Yunpeng Liu, Frank Wood, and Geoff Pleiss. Layerwise proximal replay: A proximal point method for online continual learning. 2024.